

Corso Blazor Moderno (.NET 10)

Benvenuti

0.1 Chi sono

- **Nome e Cognome:** Daniele Gastaldo
- **Ruolo:** Technical analyst and developer
- **Esperienza:** 5+ su tecnologie Microsoft .NET.
- **Contatti:**
 -  daniele.gastaldo@reteinformatica.com

0.2 Organizzazione del Corso

- **Durata Totale:** 14 ore (2 Giornate).
- **Metodologia:** Teoria affiancata da laboratori Pratici.
- **Giorno 1:**
 - Architettura, Componenti, Ciclo di Vita, Form, Layout, librerie di componenti e routing
- **Giorno 2:**
 - JS Interop, Sicurezza, Stato e PWA.
- **Obiettivo:** Costruire una Web App completa e performante.

Esercitazione Modulo 0: Setup

Obiettivo: Configurare l'ambiente

Task 1: Verifica prerequisiti

1. Aprire il terminale/prompt dei comandi.
2. Digitare `dotnet --version`.
3. Assicurarsi che la versione sia **8.0.xxx** o superiore.
4. Installare globalmente Entity Framework Tools (della versione appropriata) col comando `dotnet tool install --global dotnet-ef`.
5. Verificare la versione installata con il comando `dotnet ef`

Esercitazione Modulo 0: Primi Passi

Task 2: Creazione del Progetto

1. Creazione Progetti:

- Apri VS 2022/2026 -> Nuovo Progetto -> **Blazor Web App** -> Nome:
`SmartHub.App`
 - **Render Mode:** Auto (Server/WASM)
 - **Location:** Per page.
- Click destro sulla Soluzione -> Aggiungi -> **Razor Class Library** -> Nome:
`SmartHub.UI` .
- Click destro sulla Soluzione -> Aggiungi -> **Class Library** -> Nome:
`SmartHub.Data` .

2. Reference (Cruciale!):

Esercitazione Modulo 0: Analisi Struttura

Esplorazione della soluzione generata

1. Analisi Progetto Server:

- `Program.cs` : Nota i servizi `AddInteractiveServerComponents` e `AddInteractiveWebAssemblyComponents` .
- `App.razor` : Il file che definisce l'HTML di base (head, body).

2. Analisi Progetto Client:

- Dove vivono i componenti che vengono scaricati nel browser.

3. Il file `Routes.razor`:

- Configurazione del router per trovare i componenti corretti.

Modulo 1

Introduzione e Architettura

1.0 Contenuti

- Cos'è Blazor?
- WebAssembly e .NET nel Browser.
- Hosting Models (Static, Server, WASM, Auto).
- Il concetto di Model.

1.1. Cos'è Blazor?

- Framework per la creazione di interfacce web interattive.
- **C# al posto di JavaScript:** Sfrutta le competenze .NET esistenti.
- **Condivisione Codice:** Usa lo stesso "Model" su Client e Server.
- **Ecosistema:** Accesso a tutte le librerie NuGet compatibili con .NET Standard/Core.

1.2. WebAssembly (WASM) 101

- **Definizione:** Formato binario per codice eseguibile nei browser moderni.
- **Performance:** Esecuzione a velocità quasi nativa.
- **Standard aperto:** Supportato da Chrome, Edge, Firefox, Safari senza plugin.
- **Perché per .NET?** Consente di far girare il runtime di .NET direttamente nel browser dell'utente.

1.3. WebAssembly e.NET nel Browser

Come funziona WebAssembly?

1. Il browser scarica il **Runtime .NET** (compilato in WASM).
2. Le DLL dell'applicazione vengono scaricate ed eseguite dal runtime WASM.
3. Il codice C# manipola il DOM tramite un bridge di interoperabilità.

1.4. La Rivoluzione di .NET 8/10

- **Prima di .NET 8:** Scelta obbligata tra *Blazor Server* o *Blazor WASM*.
- **Oggi:** Blazor Web App.
- **Render Modes:** È possibile decidere come ogni singolo componente viene renderizzato (Server o Client).

1.5. Novità in .NET 10

- **Static SSR Hydration:** Transizione fluida tra caricamento statico e interattività senza ricaricare lo stato del DOM.
- **WASM Multi-threading:** Supporto ottimizzato per l'esecuzione parallela lato client.
- **Smart Components:** Introduzione di componenti pre-integrati con librerie AI (Semantic Search, Smart TextArea).
- **Hot Reload 2.0:** Modifiche al codice C# istantanee anche su strutture complesse di componenti.

1.6. Hosting Models: Static SSR

- **Static Server Rendering (SSR)**
- Il server genera l'HTML e lo invia al client (come ASP.NET MVC).
- **Pro:** SEO perfetto, velocità di caricamento, nessun overhead JS/WASM.
- **Contro:** Nessuna interattività (richiede refresh della pagina).

1.7. Hosting Models: Interactive Server

- **Precedentemente: Blazor Server**
- Usa **SignalR** (WebSockets) per gestire gli eventi.
- **Flusso:** Click sul browser -> Segnale al Server -> Calcolo C# -> Update DOM inviato al browser.
- **Vantaggi:** Accesso diretto alle risorse del server (DB, File System).

1.8. Hosting Models: Interactive WebAssembly

- **Precedentemente: Blazor WASM**
- L'app gira interamente sul browser.
- **Vantaggi:** Funziona offline, scarica il carico dal server (scalabilità).
- **Svantaggi:** "Payload" iniziale elevato (download del runtime).

1.9. Il Modello "Interactive Auto" (.NET 10 Edition)

- Il meglio dei due mondi:
 - i. **Primo caricamento:** Static SSR (istantaneo).
 - ii. **Background:** Download ottimizzato del runtime WASM.
 - iii. **Passaggio (Hydration):** In .NET 10, il passaggio non resetta più lo stato dei form o lo scroll della pagina.
- **Risultato:** Esperienza "App-like" immediata senza compromessi.

1.10. Riepilogo Render Modes

Modo	Dove gira?	Connessione	Ideale per...
Static	Server	HTTP	Pagine statiche
Server	Server	SignalR	App gestionali interne
WASM	Client	Nessuna	App pubbliche, offline
Auto	Entrambi	Dinamica	Esperienza utente top

1.11. Il concetto di "Model"

- In Blazor, il **Model** è una classe C# che rappresenta i dati.
- **Sharing:** Definisci il modello nel progetto Data (dove aggiungeremo anche il DbContext) in questo modo sarà condiviso da tutti i progetti che referenziano il progetto **Data**

```
public class IoTDevice {  
    public int Id { get; set; }  
    public string Name { get; set; } = "";  
    public bool IsOn { get; set; }  
    public double Value { get; set; }  
    public string? Description { get; set; }  
}
```

1.12. Preparazione data context

- Il DataContext è il tipo di classe utilizzata da Entity Framework per interagire col DataBase.
- Per le esercitazioni useremo SQLite, ma il concetto è estendibile ad altri tipi di DataBase.

```
public class SmartDbContext : DbContext {  
    public SmartDbContext(DbContextOptions<SmartDbContext> options) : base(options) { }  
    public DbSet<IoTDevice> Devices => Set<IoTDevice>();  
  
    protected override void OnModelCreating(ModelBuilder mb) {  
        mb.Entity<IoTDevice>().HasData(  
            new IoTDevice { Id = 1, Name = "Termostato", Value = 20, IsOn = true },  
            new IoTDevice { Id = 2, Name = "Luci LED", IsOn = false }  
        );  
    }  
}
```

1.13. Integrazione data context

- L'integrazione del DataContext avviene inserendo un segmento come questo:

```
builder.Services.AddDbContext<SmartDbContext>(opt =>  
    opt.UseSqlite("Data Source=../SmartHub.db")); // Percorso relativo alla root
```

- Poi si esegue la prima migrazione verso il DB per creare il file e le tabelle:

```
dotnet ef migrations add InitialCreate --project SmartHub.Data --startup-project SmartHub.App  
dotnet ef database update --project SmartHub.Data --startup-project SmartHub.App
```

Esercitazione Modulo 1: Sperimentare i Render Modes

1. Apri il file `Components/Pages/Home.razor`.
2. Aggiungi un pulsante con un contatore semplice:

```
<button @onclick="IncrementCount">Click me</button>
<p>Current count: @currentCount</p>

@code {
    private int currentCount = 0;
    private void IncrementCount() => currentCount++;
}
```

Verificare che il bottone effettivamente incrementa il conteggio solo in modalità interattiva (Server o WebAssembly)

Modulo 2: Componenti Razor

Creare l'interfaccia usando componenti più piccoli

2.1. Cos'è un Componente?

- Un'unità UI autonoma (es. un bottone, un menu, una tabella).
- File con estensione **.razor**.
- Combina markup HTML e logica C#.
- Può contenere altri componenti.

2.2. Anatomia di un Componente

- **Direttive:** `@page "/url"`, `@using`, `@inject`.
- **Markup:** HTML standard.
- **Blocco @code:** Dove risiede la logica (proprietà, metodi).
- **CSS Isolation:** File `NomeComponente.razor.css` per stili locali.

2.3. Parametri dei Componenti

- I componenti comunicano tramite [Parameter].
- Esempio nel componente `Saluto.razor` :

```
<h3>Ciao @Nome!</h3>

@code {
    [Parameter]
    public string Nome { get; set; } = "Ospite";
}
```

- Utilizzo:

```
<Saluto Nome="Daniele" />
```

2.4. Comunicazione Child -> Parent

- Si usa EventCallback.

Il figlio espone un evento, il padre lo "ascolta".

```
// Nel Figlio  
[Parameter] public EventCallback<string> OnAction { get; set; }  
  
// Nel Padre  
<Figlio OnAction="GestisciEvento" />
```

Esercitazione modulo 3: Componente Card

```
<div class="card @(Device.IsOn ? "border-primary" : "")">
  <div class="card-body">
    <h5>@Device.Name</h5>
    <button class="btn @(Device.IsOn ? "btn-success" : "btn-outline-secondary)"
      @onclick="Toggle"> @(Device.IsOn ? "ON" : "OFF") </button>
  </div>
</div>

@code {
  [Parameter] public IoTDevice Device { get; set; } = default!;
  [Parameter] public EventCallback OnToggle { get; set; }
  private async Task Toggle() => await OnToggle.InvokeAsync();
}
```

Esercitazione modulo 3: Uso del nuovo componente

```
@page "/"
@Inject SmartDbContext Db
@rendermode InteractiveAuto

<div class="row">
    @foreach (var d in devices) {
        <div class="col-md-4">
            <DeviceCard Device="d" OnToggle="() => Save(d)" />
        </div>
    }
</div>

@code {
    List<IoTDevice> devices = new();
    protected override async Task OnInitializedAsync() => devices = await Db.Devices.ToListAsync();
    async Task Save(IoTDevice d) { d.IsOn = !d.IsOn; await Db.SaveChangesAsync(); }
}
```

Modulo 3: Ciclo di Vita del Componente

Gestire il comportamento nel tempo

1. Il flusso di esecuzione

Il ciclo di vita è una serie di metodi che Blazor chiama automaticamente:

1. **SetParametersAsync**: Riceve i parametri dal componente padre.
2. **OnInitialized / Async**: Il componente viene creato (una sola volta).
3. **OnParametersSet / Async**: Chiamato ogni volta che i parametri cambiano.
4. **OnAfterRender / Async**: Il componente è visibile nel browser.
5. **Dispose**: Il componente viene distrutto.

2. OnInitializedAsync

- È il metodo più utilizzato.
- **Scopo:** Caricamento dati iniziale (chiamate API, database).
- **Nota:** In .NET 8 SSR può essere eseguito due volte (una sul server per l'HTML iniziale, una sul client per l'interattività).

3. OnAfterRenderAsync

- Viene eseguito **dopo** che il DOM è stato aggiornato.
- **Scopo:** Interazione con librerie JavaScript o manipolazione del DOM.
- **Parametro** `firstRender` : Permette di eseguire codice solo la prima volta che la pagina appare.

Modulo 4: Form e Validazione

Gestire l'input dell'utente

4.1. Il componente `<EditForm>`

Invece del tag HTML `<form>`, Blazor usa `EditForm`:

- **Model:** L'oggetto C# che contiene i dati.
- **OnValidSubmit:** Evento scatenato se la validazione passa.
- **OnInvalidSubmit:** Evento scatenato se ci sono errori.

4.2. Componenti di Input

Blazor mappa i campi HTML con componenti C# che supportano la validazione:

- `InputText` -> `<input type="text">`
- `InputNumber` -> `<input type="number">`
- `InputSelect` -> `<select>`
- `InputCheckbox` -> `<input type="checkbox">`

Binding: Si usa `@bind-Value="modello.Proprieta"`.

4.3. Validazione con Data Annotations

Si definiscono le regole direttamente nella classe C# (Model):

```
public class User {  
    [Required(ErrorMessage = "Il nome è obbligatorio")]  
    public string Name { get; set; }  
  
    [EmailAddress]  
    public string Email { get; set; }  
}
```

4.4. Attivare la validazione

Nel form si aggiunge

```
<DataAnnotationsValidator />
```

per attivare i controlli e

```
<ValidationMessage For="()" => modello.Proprieta" />
```

per mostrare i messaggi di errore

4.5. EditForm e Smart Components

In .NET 10, oltre ai componenti standard, abbiamo i **Smart Components**:

- **SmartTextArea**: Suggerisce completamenti basati sul contesto (AI-driven).
- **SmartComboBox**: Ricerca semantica invece di semplice ricerca testuale.
- **Validazione AI**: Possibilità di validare il tono o il senso di un testo via LLM direttamente nel model.

Esercitazione modulo 4: Creiamo un form di aggiunta di un dispositivo (1)

- Crea la pagina di creazione del nuovo dispositivo usando questo codice

```
@page "/add-device"
@Inject SmartDbContext Db

<h3>Nuovo Dispositivo IoT</h3>

<EditForm Model="newDevice" OnValidSubmit="HandleSave" FormName="AddDeviceForm">
    <DataAnnotationsValidator />

    @if (errorMessage != null)
    {
        <div class="alert alert-danger">@errorMessage</div>
    }

    <div class="mb-3">
        <label>Nome Dispositivo</label>
        <InputText @bind-Value="newDevice.Name" class="form-control" />
        <ValidationMessage For="() => newDevice.Name" />
    </div>

    <!-- Sezione Smart vista prima -->
    <div class="mb-3">
        <label>Descrizione Assistita (AI)</label>
        <InputTextArea @bind-Value="newDevice.Description" class="form-control" />
    </div>

    <button type="submit" class="btn btn-primary" disabled="@isSaving">
        @if (isSaving)
        {
            <span class="spinner-border spinner-border-sm"></span> <span>Salvataggio...</span>
        }
    </button>
</EditForm>
```


Esercitazione modulo 4: Creiamo un form di aggiunta di un dispositivo (2)

- Creiamo assieme il metodo di Handle Save

```
@code {  
    // Iniezione dei servizi necessari  
    [Inject] private SmartDbContext Db { get; set; }  
    [Inject] private DeviceStateService StateService { get; set; } // Per aggiornare il contatore globale  
  
    private IoTDevice newDevice = new();  
    private bool isSaving = false;  
    private string? errorMessage;  
  
    private async Task HandleSave()  
    {  
        // 1. Inizio operazione: mostriamo un indicatore di caricamento  
        isSaving = true;  
        errorMessage = null;  
  
        try  
        {  
            // 3. Validazione logica extra (esempio: controllo duplicati)  
            bool exists = await Db.Devices.AnyAsync(d => d.Name.ToLower() == newDevice.Name.ToLower());  
            if (exists)  
            {  
                errorMessage = "Esiste già un dispositivo con questo nome.";  
                isSaving = false;  
                return;  
            }  
  
            // 4. Persistenza su SQLite  
            Db.Devices.Add(newDevice);  
            await Db.SaveChangesAsync();  
  
            // 5. Aggiornamento dello Stato Globale (Pattern visto nel Giorno 2)  
            // Notifichiamo a tutta l'app che c'è un nuovo dispositivo  
            var totalCount = await Db.Devices.CountAsync();  
            StateService.UpdateCount(totalCount);  
        }  
        catch (Exception ex)  
        {  
            // 7. Gestione degli errori (Log e feedback visivo)  
            errorMessage = "Si è verificato un errore imprevisto durante il salvataggio.";  
            Console.WriteLine($"Errore: {ex.Message}");  
        }  
    }  
}
```

Esercitazione modulo 4: Creiamo un form di aggiunta di un dispositivo (3)

- Se volessimo utilizzare gli SmartComponents (componenti AI in grado di autocompilarsi in base al contesto), sarebbe sufficiente modificare l'InputTextArea in una **SmartTextArea** e aggiungere la proprietà `UserContext`.
- Attenzione. Gli SmartComponent richiedono una connessione a qualche servizio di AI (chiave API di OpenAPI o, da .NET 10, anche modelli locali come Llama-3 o Phi-3 che girano localmente sulla macchina tramite Ollama)

```
@page "/add-device"
[inject SmartDbContext Db]

<h3>Nuovo Dispositivo IoT</h3>

<EditForm Model="newDevice" OnValidSubmit="HandleSave" FormName="AddDeviceForm">
  <DataAnnotationsValidator />

  @if (errorMessage != null)
  {
    <div class="alert alert-danger">@errorMessage</div>
  }

  <div class="mb-3">
    <label>Nome Dispositivo</label>
    <InputText @bind="newDevice.Name" class="form-control" />
    <ValidationMessage For="() => newDevice.Name" />
  </div>
```

Modulo 5: Gestione del Layout

Organizzare la struttura dell'applicazione

5.1. Cos'è un Layout in Blazor?

- Un layout è un componente speciale che definisce la struttura comune (es. Header, Sidebar, Footer).
- Evita la duplicazione del codice HTML in ogni pagina.
- **Requisito tecnico:** Deve ereditare da `LayoutComponentBase`.
- **Il segnaposto @Body:** Indica il punto esatto in cui verranno inseriti i contenuti delle singole pagine.

5.2. Anatomia di MainLayout.razor

```
@inherits LayoutComponentBase

<div class="page">
    <header>Mio Sito Blazor</header>

    <main>
        @Body
    </main>

    <footer>© 2024 - Corso Blazor</footer>
</div>
```

5.3 Applicare un Layout

Esistono tre modi per assegnare un layout a una pagina:

- Globale: Definito nel file `Routes.razor` (Default).
- Per Pagina: Usando la direttiva `@layout NomeLayout` in cima al file `.razor`.
- Per Cartella: Definendo il file `_Imports.razor` in una sottocartella.

5.4 Layout Annidati (Nested Layouts)

È possibile creare un layout che "avvolge" un altro layout.

Utile per aree riservate (es. un AdminLayout che vive dentro il MainLayout).

Sintassi: Il layout "figlio" deve avere `@layout MainLayout` e al suo interno usare `@Body`.

5.5 Novità .NET 8: Le Sezioni (Sections)

A volte una pagina deve inviare contenuti specifici in un punto del layout (es. un pulsante "Salva" nella Navbar).

SectionOutlet: Definito nel Layout (il contenitore).

```
<SectionOutlet SectionName="top-bar" />
```

SectionContent: Definito nella Pagina (il contenuto).

```
<SectionContent SectionName="top-bar"> ... </SectionContent>
```


5.6 CSS Isolation e Layout

Anche i layout possono avere il loro file `.razor.css`.

È la best practice per gestire la griglia (Grid/Flexbox) della struttura principale senza influenzare i singoli componenti interni.

Esercitazione Modulo 5: Layout Personalizzato

- Obiettivo: Creare un layout specifico per una sezione del sito.
Crea un file `Components/Layout/EmptyLayout.razor`.
Fallo ereditare da `LayoutComponentBase`.
Crea una nuova pagina `Login.razor`.
Applica il layout vuoto alla pagina di login usando `@layout EmptyLayout`.
Bonus: Usa una `Section` per cambiare il titolo della scheda del browser o della Navbar dinamicamente dalla pagina Home

Modulo 7: Creazione di Librerie

Razor Class Libraries (RCL)

7.1. Cosa sono le Razor Class Libraries?

- Sono progetti che permettono di pacchettizzare componenti UI, pagine, immagini e stili CSS.
- **Obiettivo:** Condividere componenti tra diversi progetti Blazor (es. un progetto Server e uno WebAssembly) o tra diverse applicazioni della stessa azienda.
- Si compilano in file `.dll` che possono essere distribuiti anche tramite **NuGet**.

7.2. Perché creare una libreria?

- **DRY (Don't Repeat Yourself):** Scrivi il componente "Bottone Aziendale" una sola volta.
- **Standardizzazione:** Garantire che tutte le app abbiano lo stesso look & feel.
- **Isolamento:** Testare i componenti UI separatamente dalla logica di business dell'app principale.

7.3. Asset Statici nelle Librerie

Le RCL hanno una cartella speciale chiamata `wwwroot` :

- Puoi includere file CSS, immagini e file JavaScript.
- **Come richiamarli nell'app principale?**

Il percorso segue lo schema:

```
_content/[NomeDellaLibreria]/[PercorsoFile]
```

- Esempio:

```
<link href="_content/MiaLibreriaUI/styles.css" rel="stylesheet" />
```

7.4. Componenti "Generic" e Reusability

Nelle librerie spesso si usano i **Generic**:

```
@typeparam TItem

<ul>
    @foreach (var item in Items)
    {
        @ItemTemplate(item)
    }
</ul>

@code {
    [Parameter] public RenderFragment<TItem> ItemTemplate { get; set; }
    [Parameter] public IReadOnlyList<TItem> Items { get; set; }
}
```

Esercitazione Modulo 7: Creazione RCL

- Obiettivo: Creare un componente "Card" riutilizzabile in una libreria esterna.
Aggiungi Progetto: Nella tua soluzione, aggiungi un nuovo progetto di tipo "Razor Class Library". Chiamalo SharedUI.Components.
Crea Componente: Sposta (o crea) un componente CustomCard.razor all'interno della libreria.
Referenzia: Nel progetto Blazor principale, aggiungi il riferimento al progetto SharedUI.Components.
Utilizza: In _Imports.razor dell'app principale aggiungi @using SharedUI.Components e usa il componente in una pagina

Modulo 8: Routing Avanzato

Routing Avanzato

8.1. Navigare tra i componenti

8.2 La direttiva @page

Ogni componente con una direttiva @page "/percorso" diventa una pagina raggiungibile.

È possibile definire più rotte per lo stesso componente:

```
@page "/prodotti"  
@page "/lista-prodotti"
```

8.3 Parametri delle Rotte

I parametri vengono estratti dall'URL e mappati su proprietà C#:

```
@page "/dettaglio/{Id:int}"  
  
<p>Stai visualizzando l'elemento: @Id</p>  
  
@code {  
    [Parameter] public int Id { get; set; }  
}
```

- Vincoli: È possibile forzare il tipo di dato, ad esempio {Id:int}, {Id:guid}, {Name:alpha}

8.4 NavigationManager

Il servizio per gestire la navigazione via codice:

- `NavigateTo("url")`: Sposta l'utente.
- `Uri`: Ottiene l'indirizzo attuale.
- `ToAbsoluteUri("url")`: Converte URL relativi in assoluti.
- `Query String`: Permette di leggere i parametri dopo il ? nell'URL (es: `/search?term=blazor`).

8.5 Bloccare la navigazione

In .NET 8/10 è possibile intercettare il cambio di pagina (utile per form non salvati):

Uso del componente

```
<NavigationLock />.
```

Proprietà `ConfirmExternalNavigation`: Mostra un popup se l'utente prova a chiudere il tab o cambiare sito.

8.6 NavigationManager in .NET 10

- **State Persistence:** In .NET 10 è più semplice passare oggetti complessi tra rotte senza usare query string.
- **Enhanced Navigation:** Migliorato il supporto per il "Back" del browser nelle app che mescolano SSR e WASM.
- **Cancellazione navigazione:** API più pulita per intercettare e bloccare lo spostamento se un'operazione asincrona è in corso.

Esercitazione modulo 8: Routing

- Abbiamo già impostato una nuova page a cui navigare nel modulo 4.
- A questo punto inseriamo un pulsante nella nostra dashboard che ci consenta di aggiungere il dispositivo, ridirgendoci al form di aggiunta.
- Verifichiamo che al click del bottone ci si presenta la pagina di aggiunta.
- Verifichiamo anche come il NavigationManager gestisca l'evento di navigazione alla pagina precedente mestrando il messaggio di conferma nel caso in cui il form sia stato modificato.

Conclusioni e Q&A

Grazie per la partecipazione!

Daniele Gastaldo

-  daniele.gastaldo@reteinformatica.com